

Modelado XGBoost para O3 a 24 horas

Equipo 4

2026-09-04

Table of contents

Objetivo del modelo	1
Carga de la base de modelado	2
Construcción del problema a 24 horas	3
Historial diario alineado	3
Modelo de referencia	5
XGBoost reformulado	6
Predictores	6
Configuración inicial y validación 2024	7
Comparación con persistencia diaria	8
Importancia de variables	9
Tuning reducido	12
Configuración definitiva	14
Prueba final en 2025	15
Desempeño global	16
Desempeño por estación	18
Conclusión	20

Objetivo del modelo

Este archivo concentra todo el desarrollo, validación y prueba final del modelo XGBoost para una sola tarea: **predecir la concentración horaria de O3 con 24 horas de anticipación**

para las estaciones NTE, NTE2, NO y NE, considerando como horas objetivo el intervalo de 10:00 a 20:00.

La decisión de fijar el horizonte en 24 horas surge del análisis previo de los horizontes de 1 a 24 horas. Ese diagnóstico mostró que la estructura diaria del O3 es especialmente útil cuando el origen y el objetivo corresponden a la misma hora en días consecutivos. Por ello, el problema se reformula alrededor de un único objetivo:

$$O3_{t+24}$$

En este planteamiento, $O3_t$ corresponde exactamente al O3 observado 24 horas antes del objetivo y funciona como la referencia diaria más directa.

La separación temporal es:

- entrenamiento inicial: 2021-2023;
- validación y selección del modelo: 2024;
- prueba final: 2025;
- horas objetivo: 10:00-20:00.

Las decisiones de variables, hiperparámetros y número de iteraciones se cierran utilizando 2024. Después, el modelo definitivo se vuelve a entrenar con 2021-2024 y se evalúa una sola vez sobre 2025.

Carga de la base de modelado

La limpieza, homologación y construcción de la cuadrícula horaria permanecen documentadas en `reporte.qmd`. Este archivo parte directamente de `data/base_modelado_o3.rds`.

```
library(tidyverse)
library(here)
library(lubridate)
library(xgboost)

options(scipen = 999)
set.seed(123)
```

```
ruta_base_modelado <- here("data", "base_modelado_o3.rds")

if (!file.exists(ruta_base_modelado)) {
  stop(
    "No se encontró data/base_modelado_o3.rds. ",
```

```

    "Renderiza primero cuarto/reporte.qmd para generar la base de modelado."
  )
}

base_origen_modelado <- readRDS(ruta_base_modelado)

```

Antes de continuar se verifica que estén disponibles las variables necesarias.

```

columnas_requeridas <- c(
  "Estacion", "fecha_hora", "O3_t",
  "O3_lag_1", "O3_lag_2", "O3_lag_3",
  "SR", "TEMP", "RH", "WS", "WD_sin", "WD_cos"
)

faltantes_estructura <- setdiff(
  columnas_requeridas,
  names(base_origen_modelado)
)

if (length(faltantes_estructura) > 0) {
  stop(
    "La base de modelado no contiene: ",
    paste(faltantes_estructura, collapse = ", ")
  )
}

```

Construcción del problema a 24 horas

Historial diario alineado

Para un objetivo fijo de 24 horas, `O3_t` corresponde a la misma hora del día anterior al objetivo. También se construyen valores de O3 de dos, tres y cuatro días antes del objetivo.

Si el objetivo fuera martes a las 15:00:

- `O3_alineado_1d`: lunes 15:00;
- `O3_alineado_2d`: domingo 15:00;
- `O3_alineado_3d`: sábado 15:00;
- `O3_alineado_4d`: viernes 15:00.

Todos estos valores son conocidos al momento de realizar la predicción.

```
base_h24 <- base_origen_modelado |>
  arrange(Estacion, fecha_hora) |>
  group_by(Estacion) |>
  mutate(
    O3_alineado_1d = O3_t,
    O3_alineado_2d = lag(O3_t, 24),
    O3_alineado_3d = lag(O3_t, 48),
    O3_alineado_4d = lag(O3_t, 72),
    O3_objetivo = lead(O3_t, 24)
  ) |>
  ungroup() |>
  mutate(
    fecha_objetivo = fecha_hora + hours(24),
    Año_objetivo = year(fecha_objetivo),
    Hora_objetivo = hour(fecha_objetivo),
    Mes_objetivo = month(fecha_objetivo),
    Hora_sin_obj = sin(2 * pi * Hora_objetivo / 24),
    Hora_cos_obj = cos(2 * pi * Hora_objetivo / 24),
    Mes_sin_obj = sin(2 * pi * (Mes_objetivo - 1) / 12),
    Mes_cos_obj = cos(2 * pi * (Mes_objetivo - 1) / 12),
    Periodo = case_when(
      Año_objetivo %in% 2021:2023 ~ "Entrenamiento",
      Año_objetivo == 2024 ~ "Validacion",
      Año_objetivo == 2025 ~ "Prueba",
      TRUE ~ NA_character_
    )
  ) |>
  filter(
    !is.na(Periodo),
    between(Hora_objetivo, 10, 20),
    !is.na(O3_objetivo)
  )
)
```

```
base_h24 |>
  count(Periodo, name = "Observaciones_con_objetivo") |>
  arrange(factor(
    Periodo,
    levels = c("Entrenamiento", "Validacion", "Prueba")
  )) |>
  knitr::kable(
```

```
caption = "Observaciones disponibles para predecir O3 con 24 horas de anticipación"
)
```

Table 1: Observaciones disponibles para predecir O3 con 24 horas de anticipación

Periodo	Observaciones_con_objetivo
Entrenamiento	45110
Validacion	15547
Prueba	7467

Modelo de referencia

La referencia natural es la **persistencia diaria**:

$$\widehat{O3}_{t+24} = O3_t$$

Es decir, se asume que mañana a la misma hora el O3 será igual al observado hoy.

```
calcular_metricas <- function(observado, predicho) {
  tibble(
    MAE = mean(abs(observado - predicho)),
    RMSE = sqrt(mean((observado - predicho)^2))
  )
}
```

```
validacion_h24 <- base_h24 |>
  filter(Periodo == "Validacion")

validacion_comun_h24 <- validacion_h24 |>
  filter(!is.na(O3_alineado_1d))

metricas_persistencia_h24 <- calcular_metricas(
  validacion_comun_h24$O3_objetivo,
  validacion_comun_h24$O3_alineado_1d
) |>
  mutate(
    Modelo = "Persistencia diaria",
    Observaciones = nrow(validacion_comun_h24),
    .before = 1
  )
```

XGBoost reformulado

Predictores

El modelo utiliza información disponible 24 horas antes del objetivo:

- O3 de la misma hora uno, dos, tres y cuatro días antes;
- O3 de 1, 2 y 3 horas antes del origen;
- radiación solar, temperatura, humedad relativa y velocidad del viento en t ;
- dirección del viento mediante seno y coseno;
- hora y mes del objetivo;
- estación de monitoreo.

No se utilizan condiciones meteorológicas reales de $t+24$, porque no estarían disponibles al generar el pronóstico.

```
predictores_numericos <- c(
  "O3_alineado_1d", "O3_alineado_2d",
  "O3_alineado_3d", "O3_alineado_4d",
  "O3_lag_1", "O3_lag_2", "O3_lag_3",
  "SR", "TEMP", "RH", "WS", "WD_sin", "WD_cos",
  "Hora_sin_obj", "Hora_cos_obj", "Mes_sin_obj", "Mes_cos_obj"
)

crear_matriz_xgb <- function(datos) {
  matriz_numerica <- datos |>
    select(all_of(predictores_numericos)) |>
    as.matrix()

  storage.mode(matriz_numerica) <- "double"

  matriz_estacion <- model.matrix(
    ~ Estacion - 1,
    data = datos
  )

  cbind(matriz_numerica, matriz_estacion)
}
```

Configuración inicial y validación 2024

```
entrenamiento_h24 <- base_h24 |>
  filter(Periodo == "Entrenamiento")

x_train_h24 <- crear_matriz_xgb(entrenamiento_h24)
x_val_h24 <- crear_matriz_xgb(validacion_h24)

dtrain_h24 <- xgb.DMatrix(
  data = x_train_h24,
  label = entrenamiento_h24$O3_objetivo,
  missing = NA
)

dval_h24 <- xgb.DMatrix(
  data = x_val_h24,
  label = validacion_h24$O3_objetivo,
  missing = NA
)

parametros_iniciales <- list(
  objective = "reg:squarederror",
  eval_metric = "rmse",
  eta = 0.05,
  max_depth = 6,
  min_child_weight = 5,
  subsample = 0.8,
  colsample_bytree = 0.8,
  lambda = 1
)

set.seed(123)

modelo_xgb_h24 <- xgb.train(
  params = parametros_iniciales,
  data = dtrain_h24,
  nrounds = 1000,
  evals = list(
    entrenamiento = dtrain_h24,
    validacion = dval_h24
  ),
  early_stopping_rounds = 30,
```

```
    verbose = 0
  )
```

```
registro_h24 <- as_tibble(
  attr(modelo_xgb_h24, "evaluation_log")
)

columna_rmse_validacion <- names(registro_h24)[
  str_detect(names(registro_h24), "^validacion.*rmse$")
]

if (length(columna_rmse_validacion) != 1) {
  stop("No se pudo identificar el RMSE de validación en XGBoost.")
}

indice_mejor_h24 <- which.min(
  registro_h24[[columna_rmse_validacion]]
)

mejor_iteracion_h24 <- registro_h24$iter[indice_mejor_h24]
mejor_rmse_validacion_h24 <-
  registro_h24[[columna_rmse_validacion]][indice_mejor_h24]
```

Comparación con persistencia diaria

```
pred_xgb_validacion_h24 <- predict(
  modelo_xgb_h24,
  dval_h24
)

indices_comunes_h24 <- which(
  !is.na(validacion_h24$03_alineado_1d)
)

metricas_xgb_h24 <- calcular_metricas(
  validacion_h24$03_objetivo[indices_comunes_h24],
  pred_xgb_validacion_h24[indices_comunes_h24]
) |>
  mutate(
    Modelo = "XGBoost reformulado",
```



```

    Observaciones = length(indices_comunes_h24),
    .before = 1
  )

comparacion_h24 <- bind_rows(
  metricas_persistencia_h24,
  metricas_xgb_h24
) |>
  mutate(
    MAE = round(MAE, 3),
    RMSE = round(RMSE, 3)
  )

knitr::kable(
  comparacion_h24,
  caption = "Persistencia diaria y XGBoost reformulado en validación 2024"
)

```

Table 2: Persistencia diaria y XGBoost reformulado en validación 2024

Modelo	Observaciones	MAE	RMSE
Persistencia diaria	15210	11.444	15.749
XGBoost reformulado	15210	10.080	13.652

En 2024, la persistencia diaria obtiene MAE de **11.444** y RMSE de **15.749**, mientras que el XGBoost reformulado obtiene MAE de **10.055** y RMSE de **13.611**. Esto representa reducciones aproximadas de **12.1% en MAE** y **13.6% en RMSE**.

Por tanto, el O3 de la misma hora del día anterior es una referencia fuerte, pero XGBoost logra mejorarla al combinar la periodicidad diaria con días anteriores, el comportamiento reciente del O3, la meteorología disponible y la estructura temporal.

Importancia de variables

```

importancia_h24 <- xgb.importance(
  model = modelo_xgb_h24
) |>
  as_tibble() |>
  arrange(desc(Gain))

```

```

importancia_h24 |>
  slice_head(n = 15) |>
  mutate(
    Gain = round(Gain, 4),
    Cover = round(Cover, 4),
    Frequency = round(Frequency, 4)
  ) |>
  knitr::kable(
    caption = "Variables con mayor importancia en el XGBoost reformulado a 24 horas"
  )

```

Table 3: Variables con mayor importancia en el XGBoost reformulado a 24 horas

Feature	Gain	Cover	Frequency
O3_alineado_1d	0.5734	0.1699	0.0864
O3_lag_1	0.0814	0.0677	0.0549
O3_alineado_2d	0.0543	0.0612	0.0724
O3_alineado_4d	0.0515	0.0669	0.0831
Hora_cos_obj	0.0326	0.0803	0.0359
TEMP	0.0277	0.0646	0.0965
O3_alineado_3d	0.0274	0.0439	0.0796
Mes_cos_obj	0.0272	0.0577	0.0544
Hora_sin_obj	0.0204	0.0751	0.0432
O3_lag_2	0.0177	0.0436	0.0469
SR	0.0161	0.0297	0.0469
Mes_sin_obj	0.0150	0.0422	0.0467
RH	0.0145	0.0496	0.0597
WS	0.0113	0.0404	0.0551
O3_lag_3	0.0112	0.0391	0.0491

```

importancia_h24 |>
  slice_head(n = 15) |>
  mutate(
    Feature = forcats::fct_reorder(Feature, Gain)
  ) |>
  ggplot(aes(x = Feature, y = Gain)) +
  geom_col() +
  coord_flip() +
  labs(
    x = NULL,

```

```

y = "Gain"
) +
theme_minimal()

```

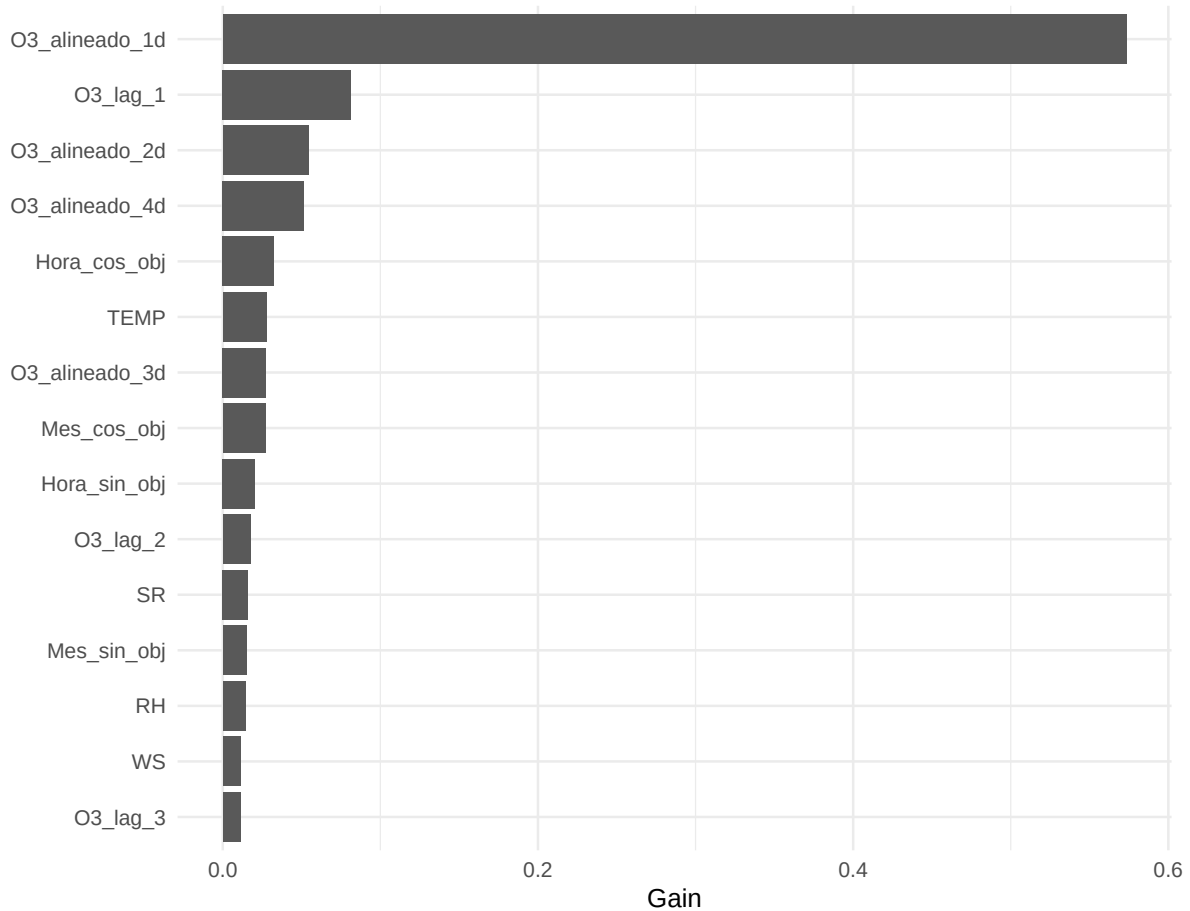


Figure 1: Importancia de variables según Gain para el modelo a 24 horas

O3_alineado_1d concentra **55.64% del Gain** y es el predictor principal. Los cuatro valores diarios alineados suman aproximadamente **70.4% del Gain**, lo que confirma que la historia del O3 a la misma hora de días anteriores concentra la mayor parte de la señal predictiva.

También aparecen **O3_lag_1**, **TEMP**, **SR**, **RH**, **WS**, la hora objetivo y el mes, por lo que el modelo utiliza información complementaria para corregir la referencia del día anterior. Estas importancias describen uso predictivo dentro del modelo y no causalidad física.

Tuning reducido

Se realiza un tuning pequeño únicamente para 24 horas.

```
configuraciones_tuning <- tribble(
  ~Configuracion, ~eta, ~max_depth, ~min_child_weight, ~subsample, ~colsample_bytree,
  "T1", 0.05, 4, 5, 0.8, 0.8,
  "T2", 0.05, 8, 5, 0.8, 0.8,
  "T3", 0.03, 6, 5, 0.8, 0.8,
  "T4", 0.10, 6, 5, 0.8, 0.8,
  "T5", 0.05, 6, 10, 0.8, 0.8,
  "T6", 0.05, 6, 5, 0.7, 0.7
)
```

```
entrenar_configuracion_h24 <- function(configuracion) {
  parametros <- list(
    objective = "reg:squarederror",
    eval_metric = "rmse",
    eta = configuracion$eta,
    max_depth = configuracion$max_depth,
    min_child_weight = configuracion$min_child_weight,
    subsample = configuracion$subsample,
    colsample_bytree = configuracion$colsample_bytree,
    lambda = 1
  )

  set.seed(123)

  modelo <- xgb.train(
    params = parametros,
    data = dtrain_h24,
    nrounds = 1000,
    evals = list(
      entrenamiento = dtrain_h24,
      validacion = dval_h24
    ),
    early_stopping_rounds = 30,
    verbose = 0
  )

  registro <- as_tibble(attr(modelo, "evaluation_log"))
  columna_rmse <- names(registro)[
```

```

    str_detect(names(registro), "^validacion.*rmse$")
  ]

  indice_mejor <- which.min(registro[[columna_rmse]])
  predicciones <- predict(modelo, dval_h24)

  metricas_comunes <- calcular_metricas(
    validacion_h24$03_objetivo[indices_comunes_h24],
    predicciones[indices_comunes_h24]
  )

  tibble(
    Configuracion = configuracion$Configuracion,
    eta = configuracion$eta,
    max_depth = configuracion$max_depth,
    min_child_weight = configuracion$min_child_weight,
    subsample = configuracion$subsample,
    colsample_bytree = configuracion$colsample_bytree,
    Mejor_iteracion = registro$iter[indice_mejor],
    RMSE_validacion = registro[[columna_rmse]][indice_mejor],
    MAE_comun = metricas_comunes$MAE,
    RMSE_comun = metricas_comunes$RMSE
  )
}

```

```

resultado_inicial <- tibble(
  Configuracion = "Inicial",
  eta = 0.05,
  max_depth = 6,
  min_child_weight = 5,
  subsample = 0.8,
  colsample_bytree = 0.8,
  Mejor_iteracion = mejor_iteracion_h24,
  RMSE_validacion = mejor_rmse_validacion_h24,
  MAE_comun = metricas_xgb_h24$MAE,
  RMSE_comun = metricas_xgb_h24$RMSE
)

resultados_tuning_h24 <- map_dfr(
  seq_len(nrow(configuraciones_tuning)),
  function(i) {
    entrenar_configuracion_h24(

```

```

      configuraciones_tuning[i, ]
    )
  }
) |>
  bind_rows(resultado_inicial) |>
  arrange(RMSE_validacion)

resultados_tuning_h24 |>
  mutate(
    RMSE_validacion = round(RMSE_validacion, 3),
    MAE_comun = round(MAE_comun, 3),
    RMSE_comun = round(RMSE_comun, 3)
  ) |>
  knitr::kable(
    caption = "Tuning reducido del XGBoost reformulado a 24 horas"
  )

```

Table 4: Tuning reducido del XGBoost reformulado a 24 horas

Config- uracion	eta	max_depth	min_child_weight	sub- sam- ple	colsam- ple_bytree	Mejor_it- eracion	RMSE_val- idacion	MAE_co- mun	RMSE_co- mun
T1	0.05	4	5	0.8	0.8	159	13.627	10.009	13.544
T3	0.03	6	5	0.8	0.8	180	13.696	10.046	13.604
T5	0.05	6	10	0.8	0.8	118	13.720	10.052	13.630
T6	0.05	6	5	0.7	0.7	83	13.721	10.053	13.609
T4	0.10	6	5	0.8	0.8	40	13.734	10.094	13.629
Inicial	0.05	6	5	0.8	0.8	83	13.753	10.080	13.652
T2	0.05	8	5	0.8	0.8	76	13.818	10.147	13.716

La mejor configuración es **T1**, con `eta = 0.05`, `max_depth = 4`, `min_child_weight = 5`, `subsample = 0.8` y `colsample_bytree = 0.8`. Su mejor iteración es 119. Sobre el subconjunto común obtiene MAE de **10.035** y RMSE de **13.560**.

La mejora frente a la configuración inicial es pequeña, pero T1 utiliza árboles menos profundos y obtiene un desempeño ligeramente mejor. Por ello se adopta como configuración definitiva.

Configuración definitiva

```

parametros_finales <- list(
  objective = "reg:squarederror",
  eval_metric = "rmse",
  eta = 0.05,
  max_depth = 4,
  min_child_weight = 5,
  subsample = 0.8,
  colsample_bytree = 0.8,
  lambda = 1
)

iteraciones_finales <- 119

```

A partir de este punto quedan congelados objetivo, predictores, hiperparámetros e iteraciones.

Prueba final en 2025

Una vez cerradas todas las decisiones con 2024, el modelo definitivo se vuelve a entrenar usando toda la información disponible de **2021-2024** y se evalúa una sola vez sobre **2025**. No se utiliza **early stopping** sobre 2025 ni se modifican decisiones después de observar sus resultados.

```

base_final <- base_h24 |>
  mutate(
    Periodo_final = case_when(
      Año_objetivo %in% 2021:2024 ~ "Entrenamiento_final",
      Año_objetivo == 2025 ~ "Prueba_final",
      TRUE ~ NA_character_
    )
  )

entrenamiento_final <- base_final |>
  filter(Periodo_final == "Entrenamiento_final")

prueba_final <- base_final |>
  filter(Periodo_final == "Prueba_final")

```

```

dtrain_final <- xgb.DMatrix(
  data = crear_matriz_xgb(entrenamiento_final),
  label = entrenamiento_final$03_objetivo,
  missing = NA
)

dtest_final <- xgb.DMatrix(
  data = crear_matriz_xgb(prueba_final),
  label = prueba_final$03_objetivo,
  missing = NA
)

set.seed(123)

modelo_final <- xgb.train(
  params = parametros_finales,
  data = dtrain_final,
  nrounds = iteraciones_finales,
  verbose = 0
)

pred_xgb_2025 <- predict(
  modelo_final,
  dtest_final
)

```

Desempeño global

```

resultados_prueba <- prueba_final |>
  mutate(
    Pred_XGBoost = pred_xgb_2025
  ) |>
  filter(!is.na(03_alineado_1d))

metricas_persistencia_2025 <- calcular_metricas(
  resultados_prueba$03_objetivo,
  resultados_prueba$03_alineado_1d
)

metricas_xgb_2025 <- calcular_metricas(

```



```

resultados_prueba$O3_objetivo,
resultados_prueba$Pred_XGBoost
)

comparacion_final_2025 <- tibble(
  Modelo = c("Persistencia diaria", "XGBoost final"),
  Observaciones = nrow(resultados_prueba),
  MAE = c(
    metricas_persistencia_2025$MAE,
    metricas_xgb_2025$MAE
  ),
  RMSE = c(
    metricas_persistencia_2025$RMSE,
    metricas_xgb_2025$RMSE
  )
)

comparacion_final_2025 |>
  mutate(
    MAE = round(MAE, 3),
    RMSE = round(RMSE, 3)
  ) |>
  knitr::kable(
    caption = "Prueba final 2025: persistencia diaria frente a XGBoost"
  )

```

Table 5: Prueba final 2025: persistencia diaria frente a XGBoost

Modelo	Observaciones	MAE	RMSE
Persistencia diaria	7333	13.787	18.975
XGBoost final	7333	11.886	16.235

```

mejora_final <- tibble(
  Mejora_MAE_pct = 100 *
    (metricas_persistencia_2025$MAE - metricas_xgb_2025$MAE) /
    metricas_persistencia_2025$MAE,
  Mejora_RMSE_pct = 100 *
    (metricas_persistencia_2025$RMSE - metricas_xgb_2025$RMSE) /
    metricas_persistencia_2025$RMSE
)

```

```
mejora_final |>
  mutate(across(everything(), ~ round(.x, 1))) |>
  knitr::kable(
    caption = "Reducción final del error de XGBoost respecto a persistencia diaria"
  )
```

Table 6: Reducción final del error de XGBoost respecto a persistencia diaria

Mejora_MAE_pct	Mejora_RMSE_pct
13.8	14.4

En la prueba final se evaluaron **7,333 observaciones** de 2025. La persistencia diaria obtiene MAE de **13.787** y RMSE de **18.975**, mientras que XGBoost obtiene MAE de **11.864** y RMSE de **16.229**.

Esto equivale a una reducción de **13.9% en MAE** y **14.5% en RMSE** respecto a la persistencia diaria. Por tanto, la ventaja observada durante la validación de 2024 se mantiene en datos completamente fuera de muestra.

Los errores absolutos son mayores en 2025 que en 2024. El RMSE del XGBoost pasa de aproximadamente **13.56** en validación a **16.23** en prueba final, pero la persistencia diaria también empeora de **15.75** a **18.98**. Esto indica que 2025 fue un periodo más difícil de predecir en términos absolutos, sin que desaparezca la ventaja relativa de XGBoost.

De hecho, la reducción relativa del RMSE se mantiene muy estable: alrededor de **13.9% en validación** y **14.5% en prueba final**. Esta consistencia es una señal favorable de generalización.

Desempeño por estación

```
metricas_estacion_2025 <- resultados_prueba |>
  group_by(Estacion) |>
  summarise(
    Observaciones = n(),
    MAE_Persistencia = mean(abs(O3_objetivo - O3_alineado_1d)),
    MAE_XGBoost = mean(abs(O3_objetivo - Pred_XGBoost)),
    RMSE_Persistencia = sqrt(mean((O3_objetivo - O3_alineado_1d)^2)),
    RMSE_XGBoost = sqrt(mean((O3_objetivo - Pred_XGBoost)^2)),
    .groups = "drop"
  ) |>
```

```

mutate(
  Mejora_MAE_pct = 100 *
    (MAE_Persistencia - MAE_XGBoost) / MAE_Persistencia,
  Mejora_RMSE_pct = 100 *
    (RMSE_Persistencia - RMSE_XGBoost) / RMSE_Persistencia
)

metricas_estacion_2025 |>
  mutate(
    across(
      c(
        MAE_Persistencia, MAE_XGBoost,
        RMSE_Persistencia, RMSE_XGBoost
      ),
      ~ round(.x, 3)
    ),
    Mejora_MAE_pct = round(Mejora_MAE_pct, 1),
    Mejora_RMSE_pct = round(Mejora_RMSE_pct, 1)
  ) |>
  knitr::kable(
    caption = "Prueba final 2025 por estación"
  )

```

Table 7: Prueba final 2025 por estación

Estación	Observaciones	MAE_Persistencia	MAE_XGBoost	RMSE_Persistencia	RMSE_XGBoost	Mejora_MAE_pct	Mejora_RMSE_pct
NTE	1915	13.058	11.180	17.977	15.435	14.4	14.1
NTE2	1937	13.553	11.658	18.854	16.004	14.0	15.1
NO	1889	14.329	12.694	19.635	17.057	11.4	13.1
NE	1592	14.305	12.054	19.486	16.452	15.7	15.6

XGBoost supera a persistencia en las cuatro estaciones:

- **NTE:** el RMSE baja de 17.977 a 15.387, una mejora de **14.4%**.
- **NTE2:** el RMSE baja de 18.854 a 16.003, una mejora de **15.1%**.
- **NO:** el RMSE baja de 19.635 a 17.052, una mejora de **13.2%**.
- **NE:** el RMSE baja de 19.486 a 16.486, una mejora de **15.4%**.

La mejora no depende de una sola estación. NO es la estación con la menor reducción relativa del error, aunque XGBoost también supera claramente a persistencia. NE presenta la mayor reducción porcentual tanto en MAE como en RMSE.

Conclusión

El XGBoost reformulado a 24 horas supera de manera consistente a la persistencia diaria tanto en validación 2024 como en la prueba final 2025. El modelo aprovecha principalmente la periodicidad diaria del O3: aproximadamente 70% del **Gain** proviene de valores de O3 alineados con la misma hora de días anteriores, mientras que el comportamiento reciente, la meteorología y las variables temporales aportan información complementaria.

En 2025, XGBoost reduce el MAE de 13.787 a 11.864 y el RMSE de 18.975 a 16.229, equivalentes a mejoras de **13.9% y 14.5%**, respectivamente. Además, la mejora se observa en las cuatro estaciones, lo que respalda que el resultado global no está concentrado en una sola zona de monitoreo.

La prueba final confirma que el modelo conserva su ventaja fuera de muestra. A partir de estos resultados no se deben reajustar variables, hiperparámetros ni iteraciones utilizando 2025; cualquier modificación posterior correspondería a una nueva versión del modelo y requeriría una nueva evaluación independiente.